

PINN_tsunami_2d

January 11, 2026

```
[ ]: ## Pawel Maczuga and Maciej Paszynski 2023

from typing import Callable
import matplotlib.pyplot as plt
import numpy as np
import torch
from torch import nn
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from functools import partial
# from google.colab import files
import time
import os
from typing import Tuple
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
from PIL import Image
import torch.nn.functional as F

[ ]: ## Parameters
# LENGTH = 1. # Domain size in x axis. Always starts at 0
LENGTH = 2. # Domain size in x axis. Always starts at 0
# TOTAL_TIME = .3 # Domain size in t axis. Always starts at 0
TOTAL_TIME = .5 # Domain size in t axis. Always starts at 0
N_POINTS = 15 # Number of in single axis
N_POINTS_PLOT = 150 # Number of points in single axis used in plotting
WEIGHT_RESIDUAL = 0.03 # Weight of residual part of loss function
WEIGHT_INITIAL = 1.0 # Weight of initial part of loss function
WEIGHT_BOUNDARY = 0.0005 # Weight of boundary part of loss function
# LAYERS = 4
LAYERS = 10
# NEURONS_PER_LAYER = 80
NEURONS_PER_LAYER = 120
EPOCHS = 1_000
# LEARNING_RATE = 0.002
LEARNING_RATE = 0.00015
GRAVITY=9.81
```

```
[ ]: class PINN(nn.Module):
    """Simple neural network accepting two features as input and returning a
    ↪single output

    In the context of PINNs, the neural network is used as universal function
    ↪approximator
    to approximate the solution of the differential equation
    """
    def __init__(self, num_hidden: int, dim_hidden: int, act=nn.Tanh()):

        super().__init__()

        self.layer_in = nn.Linear(3, dim_hidden)
        self.layer_out = nn.Linear(dim_hidden, 1)

        num_middle = num_hidden - 1
        self.middle_layers = nn.ModuleList(
            [nn.Linear(dim_hidden, dim_hidden) for _ in range(num_middle)]
        )
        self.act = act

    def forward(self, x, y, t):

        x_stack = torch.cat([x, y, t], dim=1)
        out = self.act(self.layer_in(x_stack))
        for layer in self.middle_layers:
            out = self.act(layer(out))
        logits = self.layer_out(out)

        return logits

    def device(self):
        return next(self.parameters()).device

def f(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor) -> torch.
    ↪Tensor:
    """Compute the value of the approximate solution from the NN model"""
    return pinn(x, y, t)

def df(output: torch.Tensor, input: torch.Tensor, order: int = 1) -> torch.
    ↪Tensor:
    """Compute neural network derivative with respect to input features using
    ↪PyTorch autograd engine"""
    df_value = output
    for _ in range(order):
```

```

        df_value = torch.autograd.grad(
            df_value,
            input,
            grad_outputs=torch.ones_like(input),
            create_graph=True,
            retain_graph=True,
        )[0]

    return df_value

def dfdt(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
    ↪int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, t, order=order)

def dfdx(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
    ↪int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, x, order=order)

def dfdy(pinn: PINN, x: torch.Tensor, y: torch.Tensor, t: torch.Tensor, order:
    ↪int = 1):
    f_value = f(pinn, x, y, t)
    return df(f_value, y, order=order)

```

0.1 Loss function

```

[ ]: def get_boundary_points(x_domain, y_domain, t_domain, n_points, device = torch.
    ↪device("cpu"), requires_grad=True):
    """
        .+-----+
        . ' |      . ' |
        +---+---+' |
        /   /   /   /
    y /   ,+---+---+
        /.'      /.' t
        +-----+'
        x
    """
    x_linspace = torch.linspace(x_domain[0], x_domain[1], n_points)
    y_linspace = torch.linspace(y_domain[0], y_domain[1], n_points)
    t_linspace = torch.linspace(t_domain[0], t_domain[1], n_points)

    x_grid, t_grid = torch.meshgrid( x_linspace, t_linspace, indexing="ij")
    y_grid, _       = torch.meshgrid( y_linspace, t_linspace, indexing="ij")

```

```

x_grid = x_grid.reshape(-1, 1).to(device)
x_grid.requires_grad = requires_grad
y_grid = y_grid.reshape(-1, 1).to(device)
y_grid.requires_grad = requires_grad
t_grid = t_grid.reshape(-1, 1).to(device)
t_grid.requires_grad = requires_grad

x0 = torch.full_like(t_grid, x_domain[0], requires_grad=requires_grad)
x1 = torch.full_like(t_grid, x_domain[1], requires_grad=requires_grad)
y0 = torch.full_like(t_grid, y_domain[0], requires_grad=requires_grad)
y1 = torch.full_like(t_grid, y_domain[1], requires_grad=requires_grad)

down    = (x_grid, y0,    t_grid)
up      = (x_grid, y1,    t_grid)
left    = (x0,    y_grid, t_grid)
right   = (x1,    y_grid, t_grid)

return down, up, left, right

```

```

[ ]: def get_initial_points(x_domain, y_domain, t_domain, n_points, device = torch.
    ↪device("cpu"), requires_grad=True):
    x_linspace = torch.linspace(x_domain[0], x_domain[1], n_points)
    y_linspace = torch.linspace(y_domain[0], y_domain[1], n_points)
    x_grid, y_grid = torch.meshgrid( x_linspace, y_linspace, indexing="ij")
    x_grid = x_grid.reshape(-1, 1).to(device)
    x_grid.requires_grad = requires_grad
    y_grid = y_grid.reshape(-1, 1).to(device)
    y_grid.requires_grad = requires_grad
    t0 = torch.full_like(x_grid, t_domain[0], requires_grad=requires_grad)
    return (x_grid, y_grid, t0)

```

```

[ ]: def get_interior_points(x_domain, y_domain, t_domain, n_points, device = torch.
    ↪device("cpu"), requires_grad=True):
    x_raw = torch.linspace(x_domain[0], x_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)
    y_raw = torch.linspace(y_domain[0], y_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)
    t_raw = torch.linspace(t_domain[0], t_domain[1], steps=n_points,
    ↪requires_grad=requires_grad)
    grids = torch.meshgrid(x_raw, y_raw, t_raw, indexing="ij")

    #W tym miejscu wczytujemy punkty z pliku siatki
    # Czy moglbys napisac tutaj wczytywanie punktow z pliku siatki
    x = grids[0].reshape(-1, 1).to(device)
    y = grids[1].reshape(-1, 1).to(device)
    t = grids[2].reshape(-1, 1).to(device)

```

```
return x, y, t
```

```
[ ]: class Loss:
    def __init__(
        self,
        x_domain: Tuple[float, float],
        y_domain: Tuple[float, float],
        t_domain: Tuple[float, float],
        n_points: int,
        initial_condition: Callable,
        floor: Callable,
        weight_r: float = 1.0,
        weight_b: float = 1.0,
        weight_i: float = 1.0,
        verbose: bool = False,
    ):
        self.x_domain = x_domain
        self.y_domain = y_domain
        self.t_domain = t_domain
        self.n_points = n_points
        self.initial_condition = initial_condition
        self.floor = floor
        self.weight_r = weight_r
        self.weight_b = weight_b
        self.weight_i = weight_i

    def residual_loss(self, pinn: PINN):
        x, y, t = get_interior_points(self.x_domain, self.y_domain, self.
↪t_domain, self.n_points, pinn.device())
        u = f(pinn, x, y, t)
        z = self.floor(x, y)
        loss = dfdt(pinn, x, y, t, order=2) - \
            GRAVITY * ( dfdx(pinn, x, y, t) ** 2 + \
                (u-z) * dfdx(pinn, x, y, t, order=2) + \
                dfdy(pinn, x, y, t) ** 2 + \
                (u-z) * dfdy(pinn, x, y, t, order=2)
            )
        return loss.pow(2).mean()

    def initial_loss(self, pinn: PINN):
        x, y, t = get_initial_points(self.x_domain, self.y_domain, self.
↪t_domain, self.n_points, pinn.device())
        pinn_init = self.initial_condition(x, y)
        loss = f(pinn, x, y, t) - pinn_init
        return loss.pow(2).mean()
```

```

def boundary_loss(self, pinn: PINN):
    down, up, left, right = get_boundary_points(self.x_domain, self.
↪y_domain, self.t_domain, self.n_points, pinn.device())
    x_down, y_down, t_down = down
    x_up, y_up, t_up = up
    x_left, y_left, t_left = left
    x_right, y_right, t_right = right

    loss_down = dfdy( pinn, x_down, y_down, t_down )
    loss_up = dfdy( pinn, x_up, y_up, t_up )
    loss_left = dfdx( pinn, x_left, y_left, t_left )
    loss_right = dfdx( pinn, x_right, y_right, t_right )

    return loss_down.pow(2).mean() + \
        loss_up.pow(2).mean() + \
        loss_left.pow(2).mean() + \
        loss_right.pow(2).mean()

def verbose(self, pinn: PINN, only_initial=False):
    """
    Returns all parts of the loss function

    Not used during training! Only for checking the results later.
    """
    residual_loss = self.residual_loss(pinn)
    initial_loss = self.initial_loss(pinn)
    boundary_loss = self.boundary_loss(pinn)

    final_loss = \
        self.weight_r * residual_loss + \
        self.weight_i * initial_loss + \
        self.weight_b * boundary_loss

    if only_initial:
        final_loss = \
            self.weight_r * residual_loss + \
            self.weight_i * initial_loss + \
            self.weight_b * boundary_loss # 5, 1000 i 1?, 0.0005

    return final_loss, residual_loss, initial_loss, boundary_loss

def __call__(self, pinn: PINN, only_initial=False):
    """
    Allows you to use instance of this class as if it was a function:

    ...
    >>> loss = Loss(*some_args)

```

```

        >>> calculated_loss = loss(pinn)
    ...
    """
    return self.verbose(pinn, only_initial)

```

0.2 Train function

```

[ ]: def train_model(
    nn_approximator: PINN,
    loss_fn: Callable,
    learning_rate: int = 0.01,
    max_epochs: int = 1_000
) -> PINN:

    optimizer = torch.optim.Adam(nn_approximator.parameters(), lr=learning_rate)
    loss_values = []
    residual_loss_values = []
    initial_loss_values = []
    boundary_loss_values = []

    start_time = time.time()

    for epoch in range(max_epochs):

        try:

            loss: torch.Tensor = loss_fn(nn_approximator)
            optimizer.zero_grad()
            loss[0].backward()
            optimizer.step()

            loss_values.append(loss[0].item())
            residual_loss_values.append(loss[1].item())
            initial_loss_values.append(loss[2].item())
            boundary_loss_values.append(loss[3].item())
            if (epoch + 1) % 1000 == 0:
                epoch_time = time.time() - start_time
                start_time = time.time()

                print(f"Epoch: {epoch + 1} - Loss: {float(loss[0].item()):>7f},
↳ Residual Loss: {float(loss[1].item()):>7f}, Initial Loss: {float(loss[2].
↳ item()):>7f}, Boundary Loss: {float(loss[3].item()):>7f}")

        except KeyboardInterrupt:
            break

```

```

    return nn_approximator, np.array(loss_values), np.
↪array(residual_loss_values), np.array(initial_loss_values), np.
↪array(boundary_loss_values)

```

0.3 Plotting functions

```

[ ]: def plot_solution(pinn: PINN, x: torch.Tensor, t: torch.Tensor, figsize=(8, 6),
↪dpi=100):

    fig, ax = plt.subplots(figsize=figsize, dpi=dpi)
    x_raw = torch.unique(x).reshape(-1, 1)
    t_raw = torch.unique(t)

    def animate(i):

        if not i % 10 == 0:
            t_partial = torch.ones_like(x_raw) * t_raw[i]
            f_final = f(pinn, x_raw, t_partial)
            ax.clear()
            ax.plot(
                x_raw.detach().numpy(), f_final.detach().numpy(), label=f"Time_
↪{float(t[i])}"
            )
            ax.set_ylim(-1, 1)
            ax.legend()

    n_frames = t_raw.shape[0]
    return FuncAnimation(fig, animate, frames=n_frames, interval=100,
↪repeat=False)

def plot_color(z: torch.Tensor, x: torch.Tensor, y: torch.Tensor, n_points_x,
↪n_points_t, title, figsize=(8, 6), dpi=100, cmap="viridis"):
    fig, ax = plt.subplots(figsize=figsize, dpi=dpi)
    z_raw = z.detach().cpu().numpy()
    x_raw = x.detach().cpu().numpy()
    y_raw = y.detach().cpu().numpy()
    X = x_raw.reshape(n_points_x, n_points_t)
    Y = y_raw.reshape(n_points_x, n_points_t)
    Z = z_raw.reshape(n_points_x, n_points_t)
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    c = ax.pcolormesh(X, Y, Z, cmap=cmap)
    fig.colorbar(c, ax=ax)

    return fig

```



```

def plot_3D(z: torch.Tensor, x: torch.Tensor, y: torch.Tensor, n_points_x, n_points_t, title, figsize=(8, 6), dpi=100, limit=0.2):
    fig = plt.figure(figsize=figsize)
    ax = fig.add_subplot(projection='3d')
    z_raw = z.detach().cpu().numpy()
    x_raw = x.detach().cpu().numpy()
    y_raw = y.detach().cpu().numpy()
    X = x_raw.reshape(n_points_x, n_points_t)
    Y = y_raw.reshape(n_points_x, n_points_t)
    Z = z_raw.reshape(n_points_x, n_points_t)
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.axes.set_zlim3d(bottom=-limit, top=limit)

    c = ax.plot_surface(X, Y, Z)

    x_floor = torch.linspace(0.0, LENGTH, 50)
    y_floor = torch.linspace(0.0, LENGTH, 50)
    z_floor = torch.zeros((50, 50))
    for x_idx, x_coord in enumerate(x_floor):
        for y_idx, y_coord in enumerate(y_floor):
            z_floor[x_idx, y_idx] = floor(x_coord, y_coord)
    x_floor = torch.tile(x_floor, (50, 1))
    y_floor = torch.tile(y_floor, (50, 1)).T
    f = ax.plot_surface(x_floor, y_floor, z_floor, color='green', alpha=0.7)

    return fig

def running_average(y, window=100):
    cumsum = np.cumsum(np.insert(y, 0, 0))
    return (cumsum[window:] - cumsum[:-window]) / float(window)

def plot_3D(z: torch.Tensor, x: torch.Tensor, y: torch.Tensor, n_points_x, n_points_t, title, figsize=(8, 6), dpi=100, limit=0.2):
    fig = plt.figure(figsize=figsize)
    ax = fig.add_subplot(projection='3d')
    z_raw = z.detach().cpu().numpy()
    x_raw = x.detach().cpu().numpy()
    y_raw = y.detach().cpu().numpy()
    X = x_raw.reshape(n_points_x, n_points_t)
    Y = y_raw.reshape(n_points_x, n_points_t)
    Z = z_raw.reshape(n_points_x, n_points_t)
    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.axes.set_zlim3d(bottom=-limit, top=limit)

```

```

c = ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)

# Plot Floor - Vectorized
x_f = torch.linspace(0.0, LENGTH, 50).to(z.device)
y_f = torch.linspace(0.0, LENGTH, 50).to(z.device)
X_f, Y_f = torch.meshgrid(x_f, y_f, indexing='ij')

# Flatten for floor function
xf_flat = X_f.reshape(-1, 1)
yf_flat = Y_f.reshape(-1, 1)

Z_f_flat = floor(xf_flat, yf_flat)
Z_f = Z_f_flat.reshape(50, 50).detach().cpu().numpy()

X_f_np = X_f.detach().cpu().numpy()
Y_f_np = Y_f.detach().cpu().numpy()

f = ax.plot_surface(X_f_np, Y_f_np, Z_f, color='green', alpha=0.5)

return fig

```

```

[ ]: # Initial condition
def initial_condition(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    r = torch.sqrt((x-LENGTH/2)**2 + (y-LENGTH/2)**2)
    # res = (r - 0.25) * (r < 0.25)
    # res = 2 * torch.exp(-(r)**2 * 30) + 2
    res = 1.0 * torch.exp(-(r)**2 * 30) + 2.0
    return res

# Load Topography
try:
    img = Image.open('topography.png').convert('L')
    img_data = np.array(img).astype(np.float32) / 255.0
    img_data = img_data * 1.0
    TOPOGRAPHY_TENSOR = torch.tensor(img_data).unsqueeze(0).unsqueeze(0).
    ↪to(device)
    print("Topography loaded:", TOPOGRAPHY_TENSOR.shape)
except Exception as e:
    print(f"Error loading topography: {e}")
    TOPOGRAPHY_TENSOR = torch.zeros((1, 1, 256, 256)).to(device)

def floor(x, y):
    """Get the sea floor value using grid_sample"""
    # x, y are tensors (N, 1) or scalars

    # Handle scalar inputs by viewing as tensor

```

```

if x.ndim == 0:
    x = x.view(1, 1)
    y = y.view(1, 1)

# Map [0, LENGTH] to [-1, 1]
grid_x = 2.0 * (x / LENGTH) - 1.0
grid_y = 2.0 * (y / LENGTH) - 1.0

# Grid sample expects (N_img, H_out, W_out, 2)
# We have 1 image. We treat points as H_out. W_out=1.
# grid shape: (1, N_points, 1, 2)

grid = torch.cat([grid_x, grid_y], dim=1) # (N_points, 2)
grid = grid.unsqueeze(0).unsqueeze(2) # (1, N_points, 1, 2)

# Sample
# TOPOGRAPHY_TENSOR is (1, 1, H, W)
z = F.grid_sample(TOPOGRAPHY_TENSOR.to(x.device), grid, align_corners=True)
# Output is (1, 1, N_points, 1)

z = z.view(-1, 1) # Back to (N_points, 1)

return z

```

1 Running code

1.1 Train data

```

[ ]: pinn = PINN(LAYERS, NEURONS_PER_LAYER, act=nn.Tanh()).to(device)

x_domain = [0.0, LENGTH]
y_domain = [0.0, LENGTH]
t_domain = [0.0, TOTAL_TIME]

# train the PINN
loss_fn = Loss(
    x_domain,
    y_domain,
    t_domain,
    N_POINTS,
    initial_condition,
    floor,
    WEIGHT_RESIDUAL,
    WEIGHT_INITIAL,
    WEIGHT_BOUNDARY
)

```

```

pinn_trained, loss_values, residual_loss_values, initial_loss_values,
↳boundary_loss_values = train_model(
    pinn, loss_fn=loss_fn, learning_rate=LEARNING_RATE, max_epochs=EPOCHS)

pinn = pinn.cpu()
losses = loss_fn.verbose(pinn)
print(f'Total loss: \t{losses[0]:.5f} ({losses[0]:.3E})')
print(f'Interior loss: \t{losses[1]:.5f} ({losses[1]:.3E})')
print(f'Initial loss: \t{losses[2]:.5f} ({losses[2]:.3E})')
print(f'Bondary loss: \t{losses[3]:.5f} ({losses[3]:.3E})')

# Plotting

# Loss function
average_loss = running_average(loss_values, window=100)
fig, ax = plt.subplots(figsize=(8, 6), dpi=100)
ax.set_title("Loss function (running average)")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.plot(average_loss)
ax.set_yscale('log')

```

```

[ ]: average_loss = running_average(residual_loss_values, window=100)
fig, ax = plt.subplots(figsize=(8, 6), dpi=100)
ax.set_title("Residual loss function (running average)")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.plot(average_loss)
ax.set_yscale('log')

```

```

[ ]: average_loss = running_average(initial_loss_values, window=100)
fig, ax = plt.subplots(figsize=(8, 6), dpi=100)
ax.set_title("Initial loss function (running average)")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.plot(average_loss)
ax.set_yscale('log')

```

```

[ ]: average_loss = running_average(boundary_loss_values, window=100)
fig, ax = plt.subplots(figsize=(8, 6), dpi=100)
ax.set_title("Boundary loss function (running average)")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.plot(average_loss)
ax.set_yscale('log')

```

```

[ ]: # Plotting
# Initial condition
base_dir = '.'
import os
os.makedirs(base_dir + '/img', exist_ok=True)
x, y, _ = get_initial_points(x_domain, y_domain, t_domain, N_POINTS_PLOT,
    ↳requires_grad=False)
z = initial_condition(x, y)
fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↳exact")
t_value = 0.0
t = torch.full_like(x, t_value)
z = pinn(x, y, t)
fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↳PINN")
x, y, _ = get_initial_points(x_domain, y_domain, t_domain, N_POINTS_PLOT,
    ↳requires_grad=False)
z = initial_condition(x, y)
fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, "Initial condition -
    ↳exact")
z = pinn(x, y, t)
fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"Initial condition -
    ↳pinn")

def plot(idx, t_value):
    t = torch.full_like(x, t_value)
    z = pinn(x, y, t)
    fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"PINN for t =
        ↳{t_value}")
    fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"PINN for t =
        ↳{t_value}")
    plt.savefig(base_dir + '/img/img_{:03d}.png'.format(idx))

time_values = np.arange(0, TOTAL_TIME, 0.01)
for idx, t_val in enumerate(time_values):
    plot(idx, t_value)
    z = pinn(x, y, t)
    fig = plot_color(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"PINN for t =
        ↳{t_value}")
    fig = plot_3D(z, x, y, N_POINTS_PLOT, N_POINTS_PLOT, f"PINN for t =
        ↳{t_val}")
    plt.savefig(base_dir + '/img/img_{:03d}.png'.format(idx))

time_values = np.arange(0, TOTAL_TIME, 0.01)
for idx, t_val in enumerate(time_values):

```

```
plot(idx, t_val)
```

```
[ ]: from google.colab import drive  
drive.mount('/content/drive')
```

```
[ ]: import imageio  
frames = []  
for idx in range(len(time_values)):  
    image = imageio.v2.imread(base_dir + '/img/img_{:03d}.png'.format(idx))  
    frames.append(image)  
  
imageio.mimsave('./tsunami_wave12.gif', # output gif  
                frames, # array of input frames  
                duration=0.1) # optional: frames per secon
```